

Benchmark oprav chyb

ESPTTest – oprava vnesené chyby (ESP32, ESP-IDF 5.2.6)

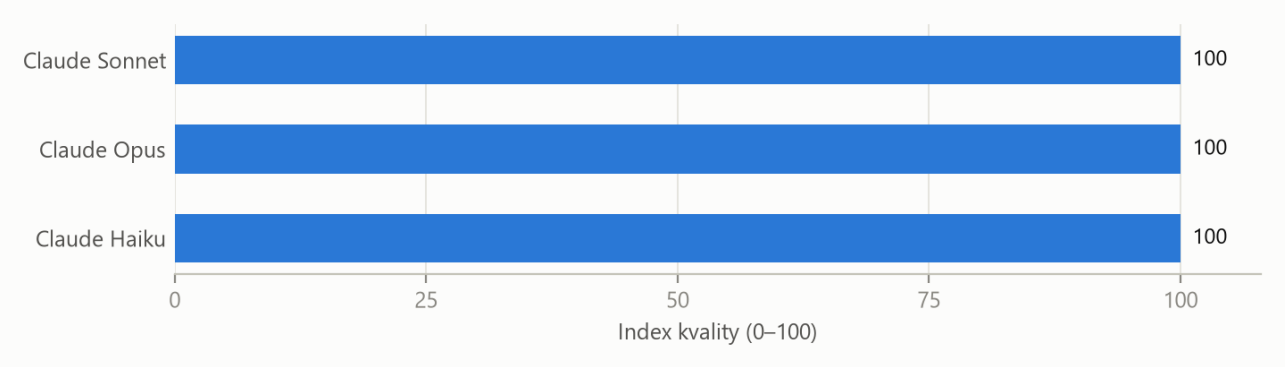
Spuštěno 2026-09-27 18:07:19 · 12 běhů · 4 chyb × 3 konfigurací × 1 opakování · Claude Code 2.1.220 (Claude Code) · OpenCode 1.18.30

Kopie funkčního firmwaru ESPTTest (commit 6fabfec) s jednou vnesenou chybou mimo OTA. Model dostane jen popis příznaku.

Poznámka: Čisté opakování 2. kola Claude modelů za 12 běhů kontaminovaných projektovým CLAUDE.md v xhard-20260927-112702. Pracovní složka mimo strom projektu (C:/esp-bench-work).

Nejlepší index 100: Claude Sonnet (95 s), Claude Opus (103 s), Claude Haiku (104 s) – při shodě seřazeno podle mediánu času.

Pořadí podle indexu kvality



Index = 100 × (0.5 × úspěšnost + 0.3 × spolehlivost + 0.2 × disciplína). Definice metrik jsou v části Metodika.

Konfigurace	Index	Úspěšnost	Spolehlivost	Shoda oprav	Disciplína	Ověřil build	Čas medián	Čas min–max	Tokeny medián	Cena celkem
Claude Sonnet	100	100 %	100 %	–	100 %	100 %	95 s	73 s–4.3 min	592 k	2.03 \$
Claude Opus	100	100 %	100 %	–	100 %	100 %	103 s	74 s–2.2 min	281 k	1.98 \$
Claude Haiku	100	100 %	100 %	–	100 %	100 %	104 s	60 s–3.1 min	533 k	0.53 \$

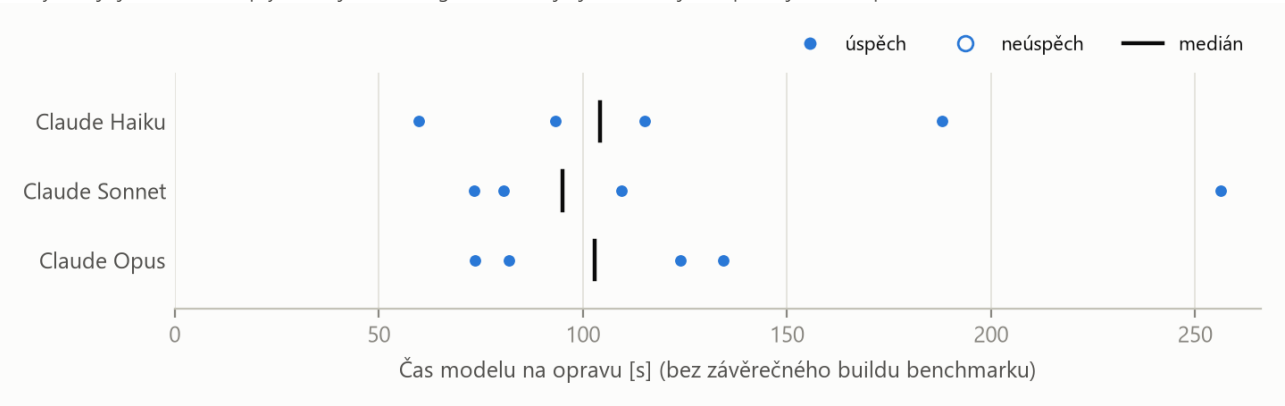
Úspěšnost podle chyb

Počet úspěšných opakování z 1. Tmavší = častější úspěch.

Konfigurace	X1 Dvě chyby: LED nezhasne + únik paměti	X2 Souběh: poškozený EventState	X3 Rollback po OTA (zpoždění startu)	X4 Nová pole RSSI a MinFreeHeap
Claude Haiku	1/1	1/1	1/1	1/1
Claude Sonnet	1/1	1/1	1/1	1/1
Claude Opus	1/1	1/1	1/1	1/1

Rychlost a její opakovatelnost

Každý bod je jeden běh. Rozptyl bodů jedné konfigurace ukazuje, jak stabilní je čas při stejném vstupu.



Chování agentů

Konfigurace	Volání nástrojů (medián)	Kroky (medián)	Ověřil build	Zamítnutá oprávnění	Chyby nástroje
Claude Haiku	15	16	100 %	3	0
Claude Sonnet	14	15	100 %	6	0
Claude Opus	12	14	100 %	1	0

Zamítnutá oprávnění = pokusy o nepovolenou akci (jiný příkaz než build, čtení mimo pracovní kopii, web).

OTA guard

Žádný běh nesáhl na chráněné části (OTA, WiFi, oddíly, motory, skript buildu).

Metodika

Každý běh dostane novou čistou kopii projektu se stejnou vnesenou chybou, stejné zadání a stejná pravidla. Liší se jen model a nástroj (harness). Běhy se střídají v pořadí opakování → chyba → konfigurace.

Model vidí jen popis příznaku, ne místo chyby. Smí číst a upravovat soubory v kopii a spustit jen build. Web, jiné příkazy a soubory mimo kopii jsou zakázané.

Po skončení modelu benchmark sám: 1) zkontroluje chráněné části (guard), 2) uloží diff, 3) obnoví skript buildu a sestaví projekt, 4) spustí kontrolu opravy pro danou chybu.

Úspěch běhu = build projde, kontrola opravy projde a guard nenajde porušení.

Úspěšnost = podíl úspěšných běhů. **Spolehlivost** (pass^k) = podíl chyb opravených ve všech 1 opakováních. **Shoda oprav** = průměrný podíl nejčastější opravy mezi opakováními (otisk diffu bez bílých znaků). **Disciplína** = podíl běhů, které něco změnily, a to bez porušení guardu a bez zbytečně velké změny (více než $\max(10, 5 \times \text{referenční oprava})$ změněných řádků); běh bez změny se nepočítá. **Ověřil build** = model sám spustil build a ten opravdu proběhl.

Čas = doba běhu modelu bez závěrečného buildu benchmarku. Tokeny = vstup (včetně cache) + výstup. Cena u Claude modelů je odhad Claude Code (předplatné se neúčtuje po tokenech), model na Sparku 0 \$.

Chyby

ID	Název	Zadání (co model dostal)
X1	Dvě chyby: LED nezhasne + únik paměti	Po poslední úpravě firmwaru jsou na desce dvě potíže: 1. Červená LED se po startu rozsvítí a už nezhasne. Má blikat: 500 ms svítí, 500 ms nesvítí. 2. Volný heap (<code>esp_get_free_heap_size</code>) klesá zhruba o 300 bajtů každých 5 sekund a deska se po několika hodinách sama restartuje. Oprav obojí.
X2	Souběh: poškozený EventState	Pole <code>EventState</code> v telemetrii má ukazovat, jestli červená LED právě svítí: <code>'blink_on'</code> , nebo <code>'blink_off'</code> . Většinou to sedí, ale zhruba jednou za pár dní přijde nesmyslná hodnota, třeba <code>'blink_onf'</code> nebo <code>'blink_of'</code> . Server ji odmítne a v dashboardu zůstane díra. Hlásí to víc desek a restart nepomáhá.
X3	Rollback po OTA (zpoždění startu)	Po každé OTA se deska zhruba po dvou minutách vrátí na předchozí firmware. V sériovém výpisu je: <code>W (402) OTA: zkusebni firmware z OTA, potvrdi se po první telemetrii E (120402) OTA: firmware se do 120 s nepotvrdil, vracím předchozí</code> Server i WiFi jsou v pořádku a starší verze firmwaru se po OTA potvrdily bez problémů. Po běžném restartu (bez OTA) deska funguje, jen se v dashboardu objeví až zhruba po dvou minutách. Dřív to bylo do 10 sekund.
X4	Nová pole RSSI a MinFreeHeap	Dashboard u desky ukazuje prázdné sloupce RSSI a MinFreeHeap. Server je čeká v objektu <code>LiveStatus</code> : - <code>'RSSI'</code> : síla signálu WiFi v dBm (celé číslo); - <code>'MinFreeHeap'</code> : nejnižší volný heap od startu, v bajtech. Deska je zatím vůbec neposílá. Když se RSSI zrovna nepodaří zjistit, pole RSSI neposílá vůbec (server by jinak uložil nesmysl). Ostatní pole telemetrie musí zůstat beze změny.

Konfigurace

Konfigurace	Nástroj	Model	Skutečně použité modely
Claude Haiku	claude-code	haiku	claude-haiku-4-5-20251001
Claude Sonnet	claude-code	sonnet	claude-haiku-4-5-20251001, claude-sonnet-5
Claude Opus	claude-code	opus	claude-haiku-4-5-20251001, claude-opus-5

Zadání (šablona)

Jsi v repozitáři firmwaru pro ESP32 (ESP-IDF v5.2.6). Přišlo hlášení chyby:

```
> {task}
```

Najdi příčinu a oprav ji.

Pravidla:

- Oprav jen tuto chybu, nic jiného neměň.
- Na OTA nesahej: `ota.c`, potvrzení firmwaru po telemetrii ani WiFi (viz README).
- Tyto části hlídá automatická kontrola a nesmí se měnit vůbec, ani drobně:
 - soubory `ota.c`, `ota.h`, `wifi.c`, `wifi.h`, `telemetry.h`, `version.h`, `device_config.h`, `partitions.csv`, `sdkconfig.defaults`, `CMakeLists.txt` a `main/CMakeLists.txt`
 - v `telemetry.c` funkce `on_http`, `handle_command`, `post`, `telemetry_task`, `telemetry_start` a konstanta `TELEMETRY_INTERVAL_S`; `payload()` musí dál posílat
 - v `main.c` pořadí `ota_boot_check` → `wifi_start` → `telemetry_start` a motory držené v nule.
- Nové soubory jen ve složce `main/`.
- Build ověříš příkazem: `sh build.sh`
- Nic nenahrávej na desku ani na server.
- Na konci stručně napiš, co bylo příčinou a co jsi změnil.

Příloha A: všechny běhy

Běh	Výsledek	Čas	Tokeny	Řádky ±	Kontrola opravy
X1-haiku-r1	úspěch	60 s	203 k	+2/-1	LED: LED_RED strida po 500 ms; pamet: payload() nic neztraci
X1-opus-r1	úspěch	74 s	238 k	+2/-1	LED: LED_RED strida po 500 ms; pamet: payload() nic neztraci
X1-sonnet-r1	úspěch	73 s	217 k	+2/-1	LED: LED_RED strida po 500 ms; pamet: payload() nic neztraci
X2-haiku-r1	úspěch	93 s	522 k	+20/-2	EventState ze stavu LED bez soubehu (pole g_event_state pod zamkem)
X2-opus-r1	úspěch	2.2 min	459 k	+11/-5	EventState ze stavu LED bez soubehu (atomicka hodnota)
X2-sonnet-r1	úspěch	110 s	561 k	+10/-6	EventState ze stavu LED bez soubehu (atomicka hodnota)
X3-haiku-r1	úspěch	3.1 min	575 k	+1/-1	pred telemetry_start() se ceka 10 s
X3-opus-r1	úspěch	2.1 min	309 k	+1/-1	pred telemetry_start() se ceka 1.25 s
X3-sonnet-r1	úspěch	4.3 min	930 k	+1/-1	pred telemetry_start() se ceka 1.25 s
X4-haiku-r1	úspěch	115 s	545 k	+8/-0	LiveStatus posila RSSI (jen pri uspechu) a MinFreeHeap
X4-opus-r1	úspěch	82 s	252 k	+9/-0	LiveStatus posila RSSI (jen pri uspechu) a MinFreeHeap
X4-sonnet-r1	úspěch	80 s	624 k	+7/-0	LiveStatus posila RSSI (jen pri uspechu) a MinFreeHeap

Příloha B: nalezené opravy

Stejně opravy (podle otisku diffu) jsou sloučené. U každé je seznam běhů, které k ní došly.

X1: Dvě chyby: LED nezhasne + únik paměti

2× (úspěch 2×): sonnet r1, opus r1

```
diff --git a/main/main.c b/main/main.c
index 9424640..d8429dc 100644
--- a/main/main.c
+++ b/main/main.c
@@ -73,8 +73,8 @@ void app_main(void)
     wifi_start();
     telemetry_start();

+   bool on = false;
     while (1) {
-       bool on = false;
         on = !on;
         gpio_set_level(LED_RED, on ? 0 : 1);
         vTaskDelay(pdMS_TO_TICKS(BLINK_MS));
diff --git a/main/telemetry.c b/main/telemetry.c
index 4ae30f3..a61ab0c 100644
--- a/main/telemetry.c
+++ b/main/telemetry.c
@@ -60,6 +60,7 @@ static char *payload(void)
     char *out = cJSON_PrintUnformatted(root);
     char *dbg = cJSON_Print(root);
     ESP_LOGD(TAG, "payload: %s", dbg);
+   cJSON_free(dbg);
+   cJSON_Delete(root);
     return out;
 }
```

1× (úspěch 1×): haiku r1

```
diff --git a/main/main.c b/main/main.c
index 9424640..e487ae4 100644
--- a/main/main.c
+++ b/main/main.c
@@ -74,7 +74,7 @@ void app_main(void)
     telemetry_start();

     while (1) {
-       bool on = false;
+       static bool on = false;
         on = !on;
         gpio_set_level(LED_RED, on ? 0 : 1);
         vTaskDelay(pdMS_TO_TICKS(BLINK_MS));
diff --git a/main/telemetry.c b/main/telemetry.c
index 4ae30f3..129f0c5 100644
--- a/main/telemetry.c
+++ b/main/telemetry.c
@@ -61,6 +61,7 @@ static char *payload(void)
     char *dbg = cJSON_Print(root);
     ESP_LOGD(TAG, "payload: %s", dbg);
     cJSON_Delete(root);
+   cJSON_free(dbg);
     return out;
 }
```

X2: Souběh: poškozený EventState

1× (úspěch 1×): haiku r1

```
diff --git a/main/main.c b/main/main.c
index 243bcf2..64a1ff6 100644
--- a/main/main.c
+++ b/main/main.c
@@ -9,6 +9,7 @@
@@ -9,6 +9,7 @@
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "driver/gpio.h"
#include "esp_log.h"
#include "nvs_flash.h"
@@ -37,6 +38,15 @@ static const char *TAG = "ESPTEST";

// Stav pro telemetrii (EventState): "blink_on", když červená prave svítí, jinak "blink_off".
char g_event_state[16] = "blink_off";
static SemaphoreHandle_t g_event_state_mutex = NULL;

+   // Thread-safe getter pro g_event_state (používá telemetry.c).
+   void get_event_state(char *buf, size_t bufsize)
+   {
+       if (g_event_state_mutex) xSemaphoreTake(g_event_state_mutex, portMAX_DELAY);
+       snprintf(buf, bufsize, "%s", g_event_state);
+       if (g_event_state_mutex) xSemaphoreGive(g_event_state_mutex);
+   }
```

```

+}

static void pins_init(void)
{
@@ -57,6 +67,8 @@ static void pins_init(void)
    gpio_set_level(LED_GREEN, 1);
    gpio_set_level(LED_BLUE, 1);
    gpio_set_level(LED_MAIN, 0);
+
+    g_event_state_mutex = xSemaphoreCreateMutex();
}

void app_main(void)
@@ -81,7 +93,9 @@ void app_main(void)
    while (1) {
        on = !on;
        gpio_set_level(LED_RED, on ? 0 : 1);
+        xSemaphoreTake(g_event_state_mutex, portMAX_DELAY);
        snprintf(g_event_state, sizeof(g_event_state), "blink_%s", on ? "on" : "off");
+        xSemaphoreGive(g_event_state_mutex);
        vTaskDelay(pdMS_TO_TICKS(BLINK_MS));
    }
}
diff --git a/main/telemetry.c b/main/telemetry.c
index 8eba0c7..eff8005 100644
--- a/main/telemetry.c
+++ b/main/telemetry.c
@@ -25,7 +25,8 @@ static const char *TAG = "TELEMETRY";
static char s_id[24];

// Stav cervene LED z main.c ("blink_on" / "blink_off").
-extern char g_event_state[16];
+// Thread-safe getter - pouziva mutex.
+extern void get_event_state(char *buf, size_t bufsize);

typedef struct {
... (zkráceno, celkem 80 řádků)
1x (úspěch 1x): sonnet r1
diff --git a/main/main.c b/main/main.c
index 243bcf2..f459e25 100644
--- a/main/main.c
+++ b/main/main.c
@@ -35,8 +35,10 @@ static const char *TAG = "ESPTEST";

#define BLINK_MS 500

-// Stav pro telemetrii (EventState): "blink_on", kdyz cervena prave sviti, jinak "blink_off".
-char g_event_state[16] = "blink_off";
+// Stav cervene LED pro telemetrii (EventState v telemetry.c). Bool misto retezce,
+// aby cteni z telemetry tasku nemohlo trefit rozepsany zapis (torn read) ze
+// snprintf a vratit necoresny stav typu "blink_onf".
+volatile bool g_led_on = false;

static void pins_init(void)
{
@@ -81,7 +83,7 @@ void app_main(void)
    while (1) {
        on = !on;
        gpio_set_level(LED_RED, on ? 0 : 1);
-        snprintf(g_event_state, sizeof(g_event_state), "blink_%s", on ? "on" : "off");
+        g_led_on = on;
        vTaskDelay(pdMS_TO_TICKS(BLINK_MS));
    }
}
diff --git a/main/telemetry.c b/main/telemetry.c
index 8eba0c7..a987557 100644
--- a/main/telemetry.c
+++ b/main/telemetry.c
@@ -1,5 +1,6 @@
#include "telemetry.h"

+#include <stdbool.h>
#include <stdio.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
@@ -24,8 +25,9 @@ static const char *TAG = "TELEMETRY";

static char s_id[24];

-// Stav cervene LED z main.c ("blink_on" / "blink_off").
-extern char g_event_state[16];
+// Stav cervene LED z main.c. Bool, ne retezec - jednotlivy zapis/cteni je
+// atomicky, takze payload() nemuze trefit rozepsany snprintf (viz main.c).
+extern volatile bool g_led_on;

typedef struct {
    char buf[1024];

```

```

@@ -54,7 +56,7 @@ static char *payload(void)
    cJSON *live = cJSON_AddObjectToObject(root, "LiveStatus");
    cJSON_AddNumberToObject(live, "Uptime", (double)(esp_timer_get_time() / 1000000));
    cJSON_AddNumberToObject(live, "ErrorCode", 0);
-   cJSON_AddStringToObject(live, "EventState", g_event_state);
+   cJSON_AddStringToObject(live, "EventState", g_led_on ? "blink_on" : "blink_off");
    // ACTIVE natvrdo: push_ota na serveru odmitne desku, o ktere si myslí,
    // ze spi, a spanek tenhle firmware neumi.
    cJSON_AddStringToObject(live, "PowerMode", "ACTIVE");

1x (úspěch 1x): opus r1
diff --git a/main/main.c b/main/main.c
index 243bcf2..13e65bf 100644
--- a/main/main.c
+++ b/main/main.c
@@ -36,7 +36,10 @@ static const char *TAG = "ESPTTEST";
#define BLINK_MS 500

// Stav pro telemetrii (EventState): "blink_on", když červená právě svítí, jinak "blink_off".
-char g_event_state[16] = "blink_off";
+// Ukazatel na literal, ne sdílený buffer: telemetrie čte z jiného úkolu a nad
+// rozepsaným bufferem by vrátila slepenec ("blink_on", "blink_off"). Zapsí
+// zároveň ukazatele je jedna instrukce, takže čte vždy celý jeden řetězec.
+const char *volatile g_event_state = "blink_off";

static void pins_init(void)
{
@@ -81,7 +84,7 @@ void app_main(void)
    while (1) {
        on = !on;
        gpio_set_level(LED_RED, on ? 0 : 1);
-       snprintf(g_event_state, sizeof(g_event_state), "blink_%s", on ? "on" : "off");
+       g_event_state = on ? "blink_on" : "blink_off";
        vTaskDelay(pdMS_TO_TICKS(BLINK_MS));
    }
}
diff --git a/main/telemetry.c b/main/telemetry.c
index 8eba0c7..8bc90d6 100644
--- a/main/telemetry.c
+++ b/main/telemetry.c
@@ -24,8 +24,9 @@ static const char *TAG = "TELEMETRY";

static char s_id[24];

-// Stav červené LED z main.c ("blink_on" / "blink_off").
-extern char g_event_state[16];
+// Stav červené LED z main.c ("blink_on" / "blink_off"). Ukazatel na literal,
+// main.c ho jen přehazuje mezi dvěma řetězci.
+extern const char *volatile g_event_state;

typedef struct {
    char buf[1024];
@@ -54,7 +55,9 @@ static char *payload(void)
    cJSON *live = cJSON_AddObjectToObject(root, "LiveStatus");
    cJSON_AddNumberToObject(live, "Uptime", (double)(esp_timer_get_time() / 1000000));
    cJSON_AddNumberToObject(live, "ErrorCode", 0);
-   cJSON_AddStringToObject(live, "EventState", g_event_state);
+   // Jedno čtení ukazatele, aby se pod rukama nezmenil během práce s ním.
+   const char *event_state = g_event_state;
+   cJSON_AddStringToObject(live, "EventState", event_state);
    // ACTIVE natvrdo: push_ota na serveru odmitne desku, o ktere si myslí,
    // ze spi, a spanek tenhle firmware neumi.
    cJSON_AddStringToObject(live, "PowerMode", "ACTIVE");

```

X3: Rollback po OTA (zpoždění startu)

```

2x (úspěch 2x): sonnet r1, opus r1
diff --git a/main/main.c b/main/main.c
index aaf04af..1b506cc 100644
--- a/main/main.c
+++ b/main/main.c
@@ -74,7 +74,7 @@ void app_main(void)
    ESP_LOGI(TAG, "ESPTTest v%s", FIRMWARE_VERSION);

    ota_boot_check();
-   vTaskDelay(POWER_SETTLE_MS * portTICK_PERIOD_MS);
+   vTaskDelay(pdMS_TO_TICKS(POWER_SETTLE_MS));
+   wifi_start();
+   telemetry_start();

1x (úspěch 1x): haiku r1
diff --git a/main/main.c b/main/main.c
index aaf04af..e4d6857 100644
--- a/main/main.c
+++ b/main/main.c
@@ -36,7 +36,7 @@ static const char *TAG = "ESPTTEST";

// Po zapnutí chvíli počkat, než se pustí WiFi: při startu bere proudovou

```

```
// spicku a na slabším zdroji deska brownoutovala. Staci ~1,25 s.
#define POWER_SETTLE_MS 1250
#define POWER_SETTLE_MS 100

static void pins_init(void)
{
```

X4: Nová pole RSSI a MinFreeHeap

1× (úspěch 1×): haiku r1

```
diff --git a/main/telemetry.c b/main/telemetry.c
index 98f4121..e8c68c6 100644
--- a/main/telemetry.c
+++ b/main/telemetry.c
@@ -6,10 +6,12 @@
#include "freertos/task.h"
#include "cJSON.h"
#include "esp_crt_bundle.h"
+#include "esp_heap_caps.h"
#include "esp_http_client.h"
#include "esp_log.h"
#include "esp_mac.h"
#include "esp_timer.h"
+#include "esp_wifi.h"

#include "device_config.h"
#include "ota.h"
@@ -55,6 +57,12 @@ static char *payload(void)
// ACTIVE natvrdo: push_ota na serveru odmitne desku, o ktere si myslí,
// ze spi, a spanek tehle firmware neumi.
cJSON_AddStringToObject(live, "PowerMode", "ACTIVE");
+ int rssi = 0;
+ if (esp_wifi_sta_get_rssi(&rssi) == ESP_OK) {
+     cJSON_AddNumberToObject(live, "RSSI", (double)rssi);
+ }
+ size_t mfh = esp_get_minimum_free_heap_size();
+ cJSON_AddNumberToObject(live, "MinFreeHeap", (double)mfh);
+ cJSON *product = cJSON_AddObjectToObject(root, "ProductInfo");
+ cJSON_AddStringToObject(product, "Název zařízení", "ESP Test");
+ char *out = cJSON_PrintUnformatted(root);
```

1× (úspěch 1×): sonnet r1

```
diff --git a/main/telemetry.c b/main/telemetry.c
index 98f4121..a3f8433 100644
--- a/main/telemetry.c
+++ b/main/telemetry.c
@@ -9,7 +9,9 @@
#include "esp_http_client.h"
#include "esp_log.h"
#include "esp_mac.h"
+#include "esp_system.h"
#include "esp_timer.h"
+#include "esp_wifi.h"

#include "device_config.h"
#include "ota.h"
@@ -52,6 +54,11 @@ static char *payload(void)
cJSON_AddNumberToObject(live, "Uptime", (double)(esp_timer_get_time() / 1000000));
cJSON_AddNumberToObject(live, "ErrorCode", 0);
cJSON_AddStringToObject(live, "EventState", "blink");
+ wifi_ap_record_t ap_info;
+ if (esp_wifi_sta_get_ap_info(&ap_info) == ESP_OK) {
+     cJSON_AddNumberToObject(live, "RSSI", ap_info.rssi);
+ }
+ cJSON_AddNumberToObject(live, "MinFreeHeap", esp_get_minimum_free_heap_size());
// ACTIVE natvrdo: push_ota na serveru odmitne desku, o ktere si myslí,
// ze spi, a spanek tehle firmware neumi.
cJSON_AddStringToObject(live, "PowerMode", "ACTIVE");
```

1× (úspěch 1×): opus r1

```
diff --git a/main/telemetry.c b/main/telemetry.c
index 98f4121..cf5a1ad 100644
--- a/main/telemetry.c
+++ b/main/telemetry.c
@@ -9,7 +9,9 @@
#include "esp_http_client.h"
#include "esp_log.h"
#include "esp_mac.h"
+#include "esp_system.h"
#include "esp_timer.h"
+#include "esp_wifi.h"

#include "device_config.h"
#include "ota.h"
@@ -55,6 +57,13 @@ static char *payload(void)
// ACTIVE natvrdo: push_ota na serveru odmitne desku, o ktere si myslí,
// ze spi, a spanek tehle firmware neumi.
cJSON_AddStringToObject(live, "PowerMode", "ACTIVE");
```

```

+ cJSON_AddNumberToObject(live, "MinFreeHeap", (double)esp_get_minimum_free_heap_size());
+ // Silu signalu zna jen pripojena STA. Kdyz ji nevrati, pole RSSI se
+ // neposle vubec, at server neulozi nulu jako platnou hodnotu.
+ wifi_ap_record_t ap;
+ if (esp_wifi_sta_get_ap_info(&ap) == ESP_OK) {
+     cJSON_AddNumberToObject(live, "RSSI", (double)ap.rssi);
+ }
+ cJSON *product = cJSON_AddObjectToObject(root, "ProductInfo");
+ cJSON_AddStringToObject(product, "Název zařízení", "ESP Test");
+ char *out = cJSON_PrintUnformatted(root);

```